

GENIFY

LECTURE 19

14-September-2025

TOPICS COVERED:

RIS

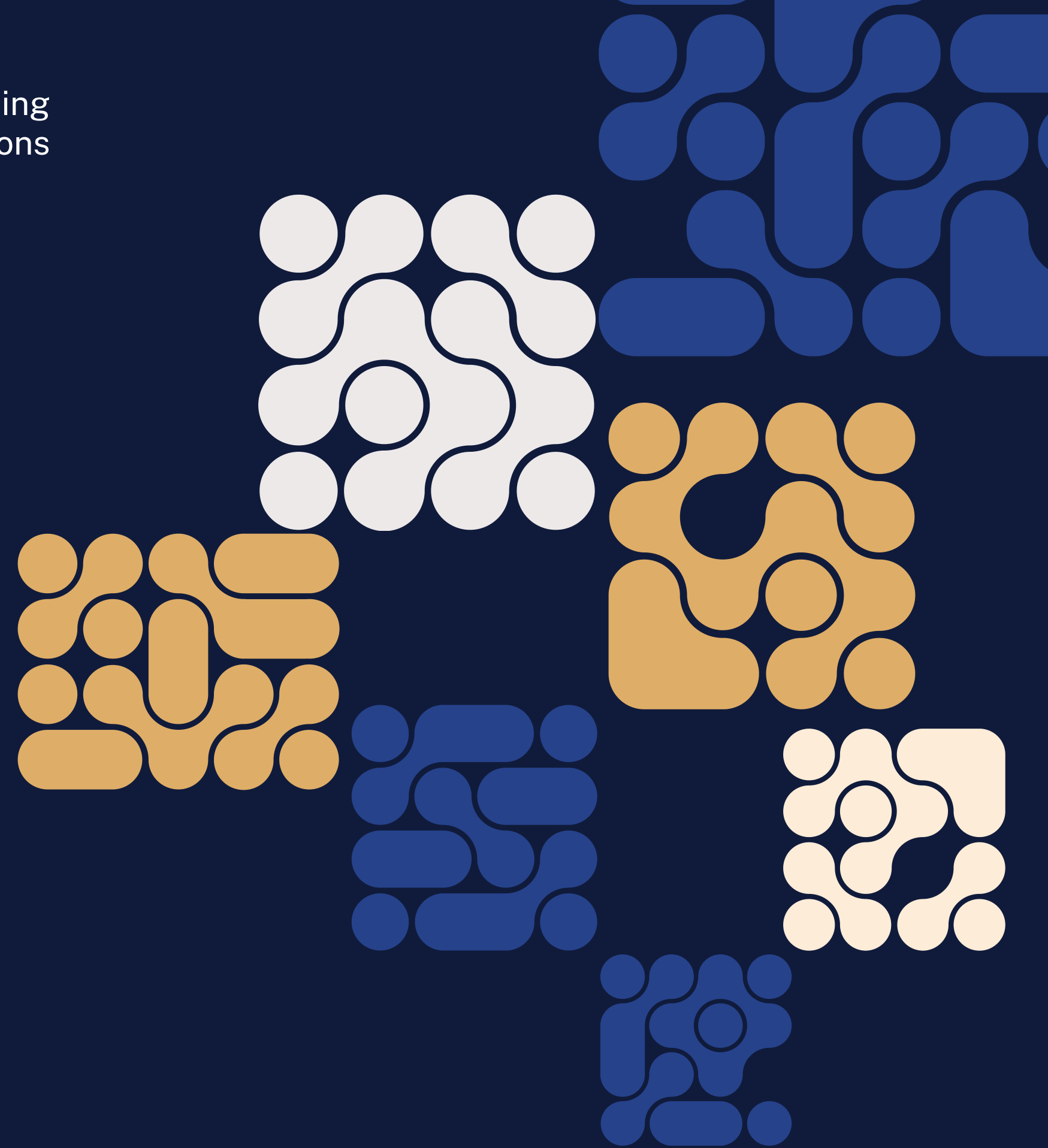
- TOKENIZATION
- VECTOR DB
- SEMANTIC SEARCH
- ECOSYSTEM OVERVIEW
- LANGCHAIN

Presenter

Shahzaib Kashif

Co-Presenter

Shayan Hassan Baig





TOKENS & TOKENIZATION

Q: What are Tokens?

In the context of LLMs, a token is a basic unit of text that the model processes. A token can represent various components of language, including:

- **Words:** In many cases, a token corresponds to a single word (e.g., "apple," "run," "quick").
- **Subwords:** For languages with a rich morphology or for more efficient processing, words may be split into subword tokens. For example, "unhappiness" might be split into "un," "happi," and "ness."
- **Punctuation:** Punctuation marks like periods, commas, and exclamation marks are also treated as individual tokens.
- **Special Tokens:** Special tokens are used for specific purposes, such as indicating the beginning or end of a sentence, padding tokens, or tokens for unknown words.

Q: Token in LLMs

LLMs have a maximum number of tokens they can process in a single request. This limit includes both the input (prompt) and the output (generated text).

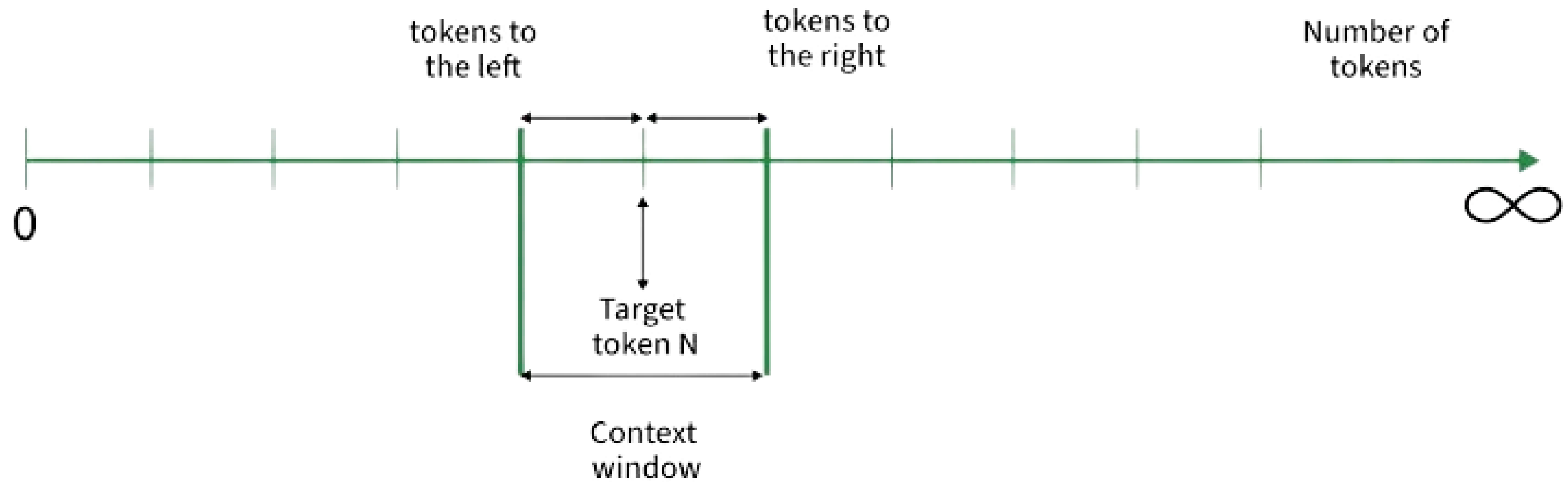
For example:

- GPT-4 has a context window of 8,192 tokens, with some versions supporting up to 32,768 tokens.
- Exceeding this limit requires truncating or splitting the text.

Q: What is Context Window?

- A context window refers to the span of text (usually in terms of tokens) that a model can consider at one time when making predictions or generating text.
- In simpler terms, it is the "lookback" or the amount of previous information that the model uses to make sense of the current input.
- LLMs, such as GPT-based models, rely heavily on context windows to predict the next token in a sequence.
- The larger the context window, the more information the model can access to understand the meaning of the text.
- However, context windows are finite, meaning that models can only consider a certain number of tokens from the input sequence before the context is truncated.

Example of a Context Window



Q: Importance of Context Window?

- **Understanding Relationships:**

- The context window helps the model understand relationships between tokens and words.
- For example, the context window allows the model to capture sentence structure, grammar, and even long-range dependencies (like subject-verb agreement).

- **Text Generation:**

- When generating text, the context window allows the model to predict the next word or token based on the input text.
- The model's ability to generate coherent and contextually relevant text relies on having enough context.

Tokenization in LLMs

- Modern LLMs typically use a form of subword tokenization (e.g., Byte Pair Encoding, WordPiece, or SentencePiece) to handle a diverse vocabulary.
- This method ensures that words or phrases are broken down into smaller, more manageable parts, allowing the model to handle a broader range of inputs without requiring an immense vocabulary.
 - **For example**, using *subword tokenization*, the word "**unbelievable**" might be split into the following tokens: "**un**," "**believ**" and "**able**".
- This way, even words that the model has never seen before can be processed effectively.

Tokenization in LLMs

- Tokenization can be likened to teaching someone a new language by starting with the alphabet, then moving on to syllables, and finally to complete words and sentences.
- This process allows for the dissection of text into parts that are easier for machines to process.
- For example, consider the sentence, "**Chatbots are helpful.**" When tokenized by words, it becomes:
 - ["Chatbots", "are", "helpful"]
- If tokenized by characters, it becomes:
 - ["C", "h", "a", "t", "b", "o", "t", "s", " ", "a", "r", "e", " ", "h", "e", "l", "p", "f", "u", "l"]
- Each approach has its own advantages depending on the context and the specific NLP task at hand.

Types of Tokenization in NLP



Types of Tokenization

Word-Based

"Machine",
"learning",
"is", "fun", "."

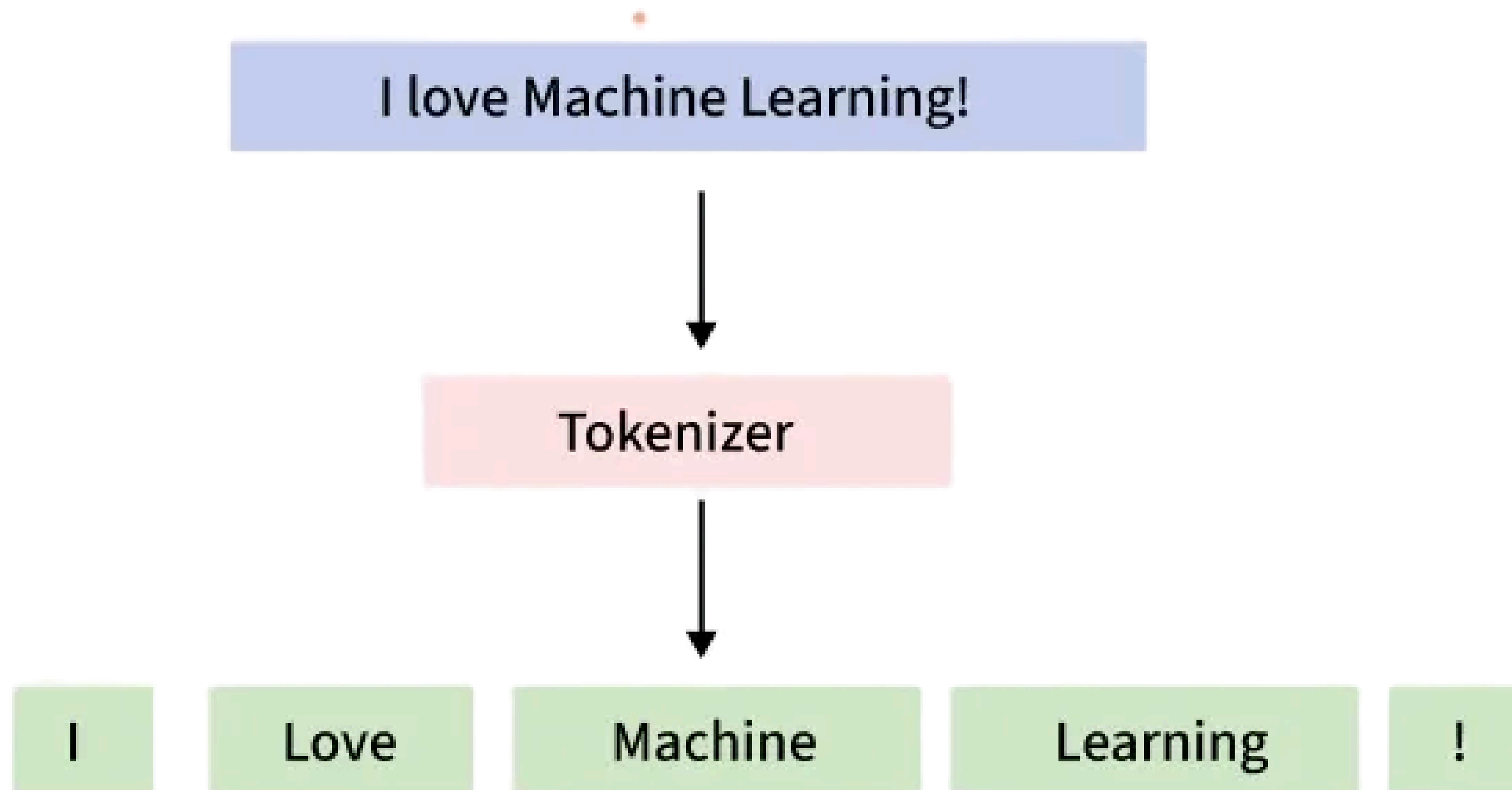
Subword-Based

"ma",
"chine",
"learn", "ing"

Character-Based

"M", "a", "c",
"h", "i", "n",
"e", "l", "e",
"a", "r", "n",
"i", "n", "g"

Tokenization in Natural Language Processing



Popular Approaches of Tokenization

**whitespace
Tokenization**

**Statistical
Tokenization**

**Transformer-
based
Tokenization**

**Rule-based
Tokenization**

**Byte-Pair
Encoding**

Techniques used in Tokenization

- **Whitespace Tokenization:** This method splits text based on the position of spaces. It doesn't handle some different cases like compound words.
- **Statistical Tokenization:** This method of tokenization uses statistical properties in text. These include frequency count of words and probability of co-occurrence.
- **Transformer-based Tokenization:** This method combines sub-words by pre-training vocabulary and also maintain consistency.
- **Rule Based Tokenization:** This approach uses manually defined if-else conditions or rules that are used to handle or tokenize regular expressions, punctuations, etc.
- **Byte-Pair Encoding:** This is a sub-word tokenization algorithm. It iteratively merges most frequent pairs of bytes or characters.

Context Windows in Transformer Models

- Transformer-based models, such as GPT, BERT, and T5, leverage self-attention mechanisms that allow the model to focus on different parts of the input sequence.
- The context window in these models is defined by the maximum number of tokens that can be processed in parallel.
 - **For example**, GPT-3 has a context window of 2048 tokens, meaning it can process up to 2048 tokens at once when making predictions or generating text.
- As the model moves through the text, the context window "slides" over the sequence, considering the most recent tokens within the window.
- This sliding window approach allows the model to maintain relevance to the most recent parts of the input while discarding older, less relevant tokens.

The following table outlines the tokenization technique and context window size of LLMs:

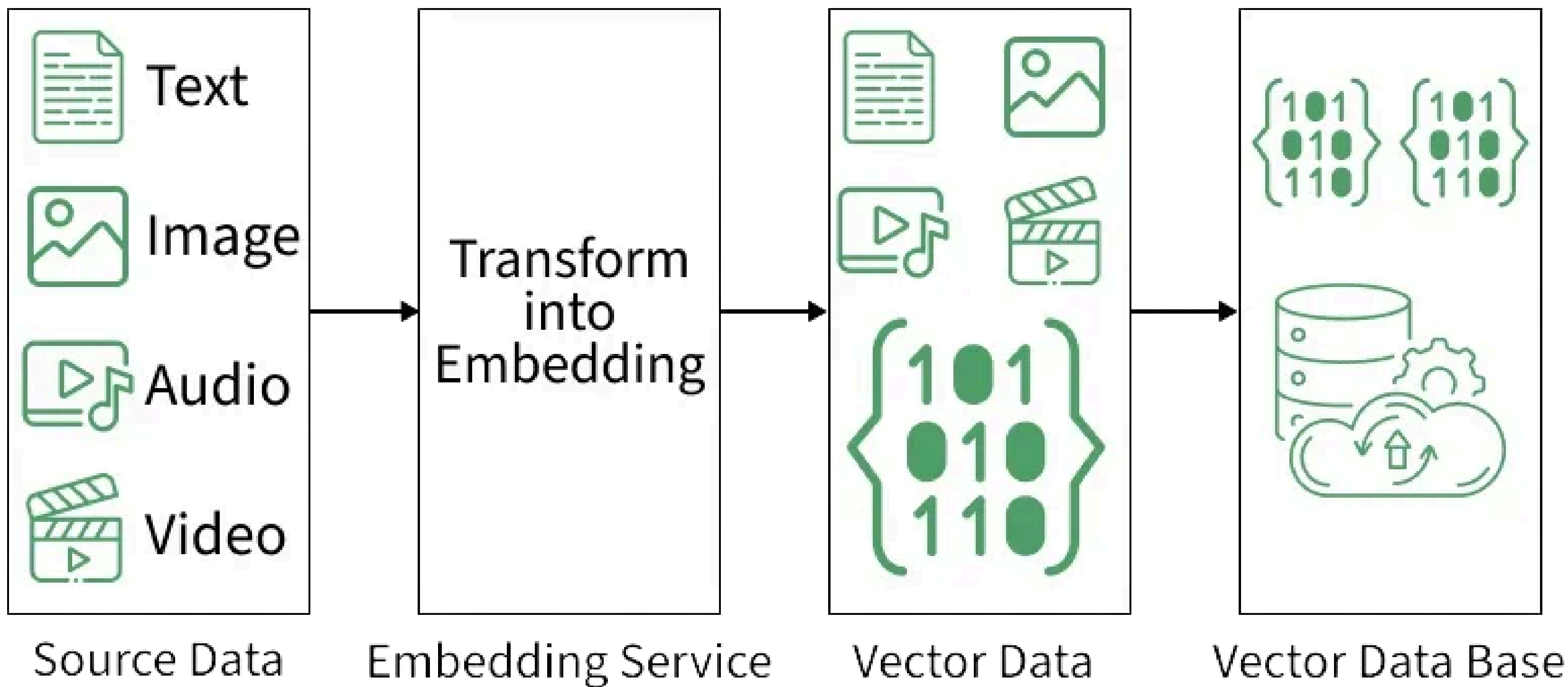
Model	Tokenization Method	Context Window Size
GPT-3	Byte Pair Encoding (BPE)	2048 Tokens
GPT-4	Byte Pair Encoding (BPE)	8192 tokens (varies by configuration)
BERT	WordPiece	512 tokens
Llama 3.1 8B	Byte Pair Encoding (BPE)	128,000 tokens
DeepSeek-R1-Distill-Llama-70B	Byte Pair Encoding (BPE)	128,000 tokens
Llama-3.3-70B-SpecDec	Byte Pair Encoding (BPE)	8,192 tokens



VECTOR DATABASE

Q: What is a Vector Database?

- A vector database is a specialized type of database designed to store, index and search high dimensional vector representations of data known as embeddings.
- Unlike traditional databases that rely on exact matches vector databases use similarity search techniques such as cosine similarity or Euclidean distance to find items that are semantically or visually similar.



Object

Vector

Task



Image

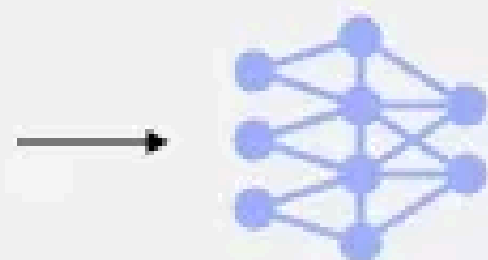


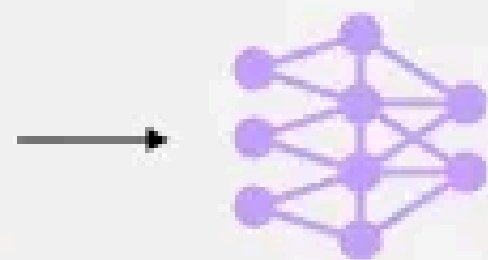
Image
transformer

→ [1.3, 0.6, 1.2, -1.3, ...] →

Object recognition,
deduplication, scene
detection, product search, ...

Happy
Birthday

Text



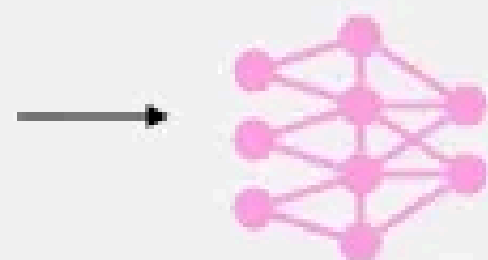
NLP
transformer

→ [0.3, -0.4, 1.2, 0.3, ...] →

Translation, understanding,
Sentiment, Question
Answering, Semantic Search, ...



Audio



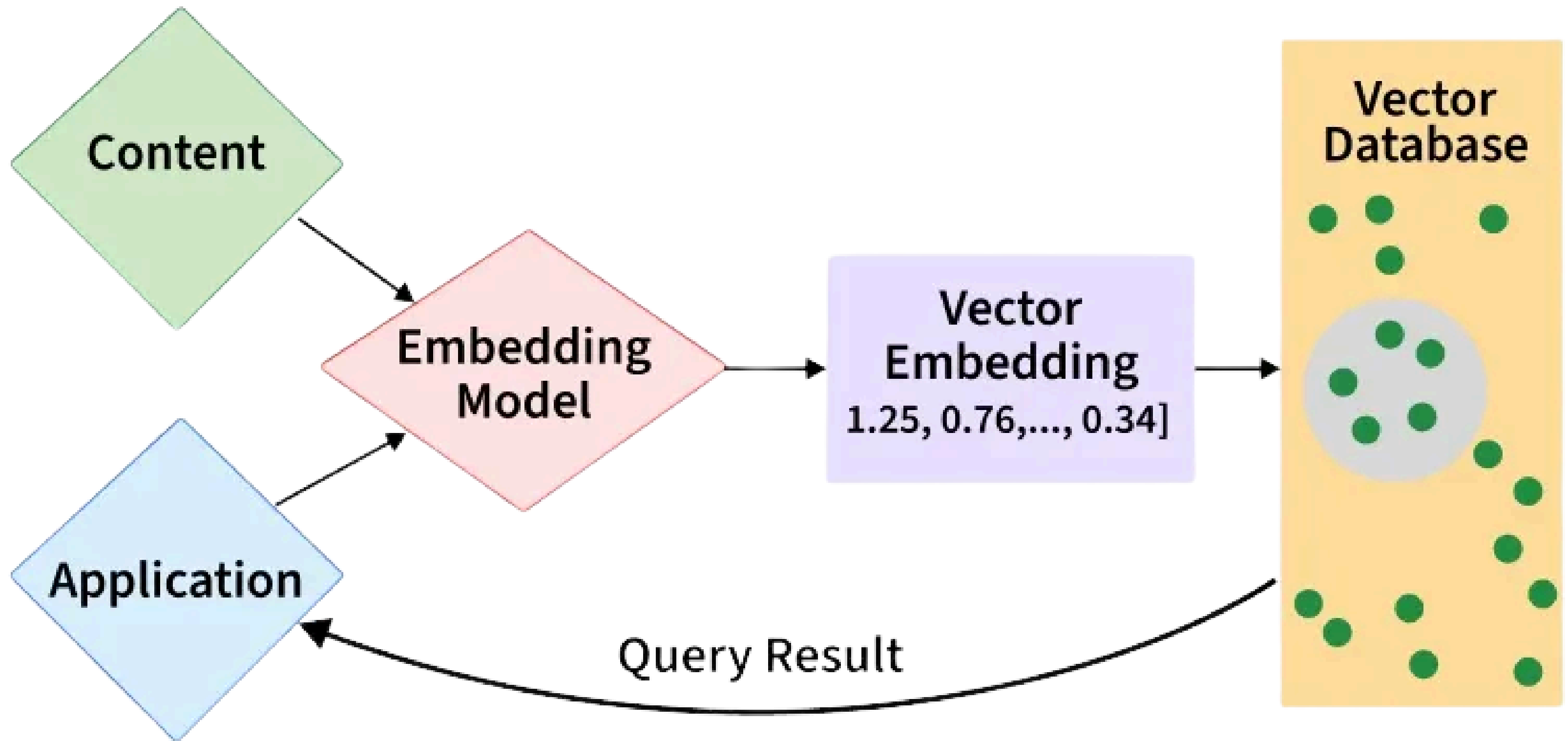
Audio
transformer

→ [1.2, -0.3, 0.7, -1.8, ...] →

Anomaly detection, speech-to-
text, music transcription,
machinery malfunction, ...

Q: Relationship of Vector DB and LLMs

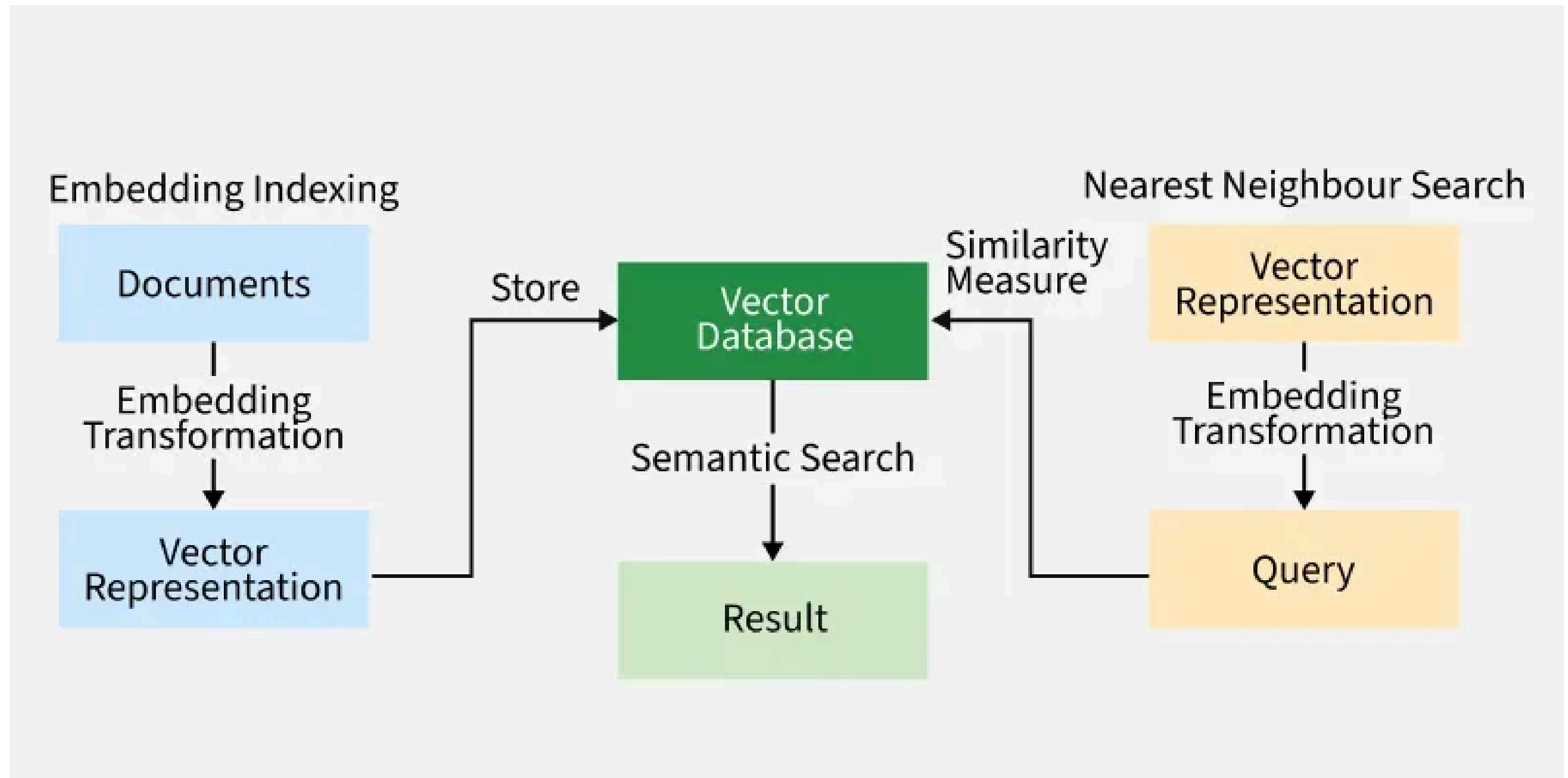
- Vector databases enhance LLMs by providing contextual, domain-specific knowledge beyond their training data.
- This integration solves key LLM limitations like illusions and outdated information by enabling:
- **Retrieval-Augmented Generation (RAG):** Retrieve relevant context before response generation.
- **Semantic search:** Find contextually similar data even without exact keywords.
- **Dynamic knowledge enrichment:** Inject real-time data into LLMs without retraining.



Real-World Example: Research Assistant

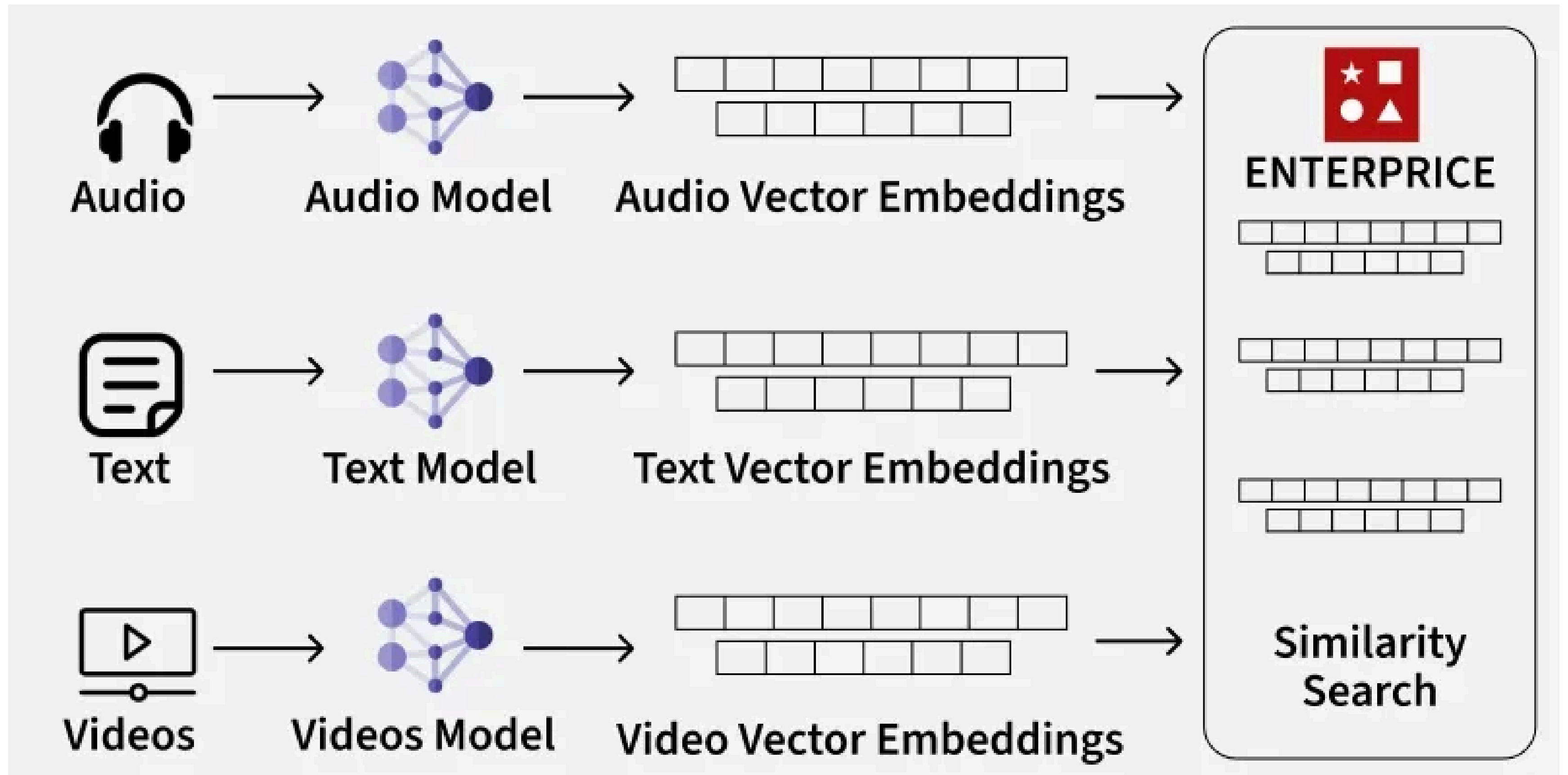
- **Scenario:** Answering "Ethical concerns in AI surveillance"
- **Retrieval:** Vector DB returns academic papers on AI ethics
- **Augmentation:** LLM synthesizes papers into response
- **Output:** "Key ethical concerns include algorithmic bias in facial recognition (Smith et al. 2023), lack of transparency in predictive policing systems (IEEE 2024) and..."

Implementing a Semantic Search



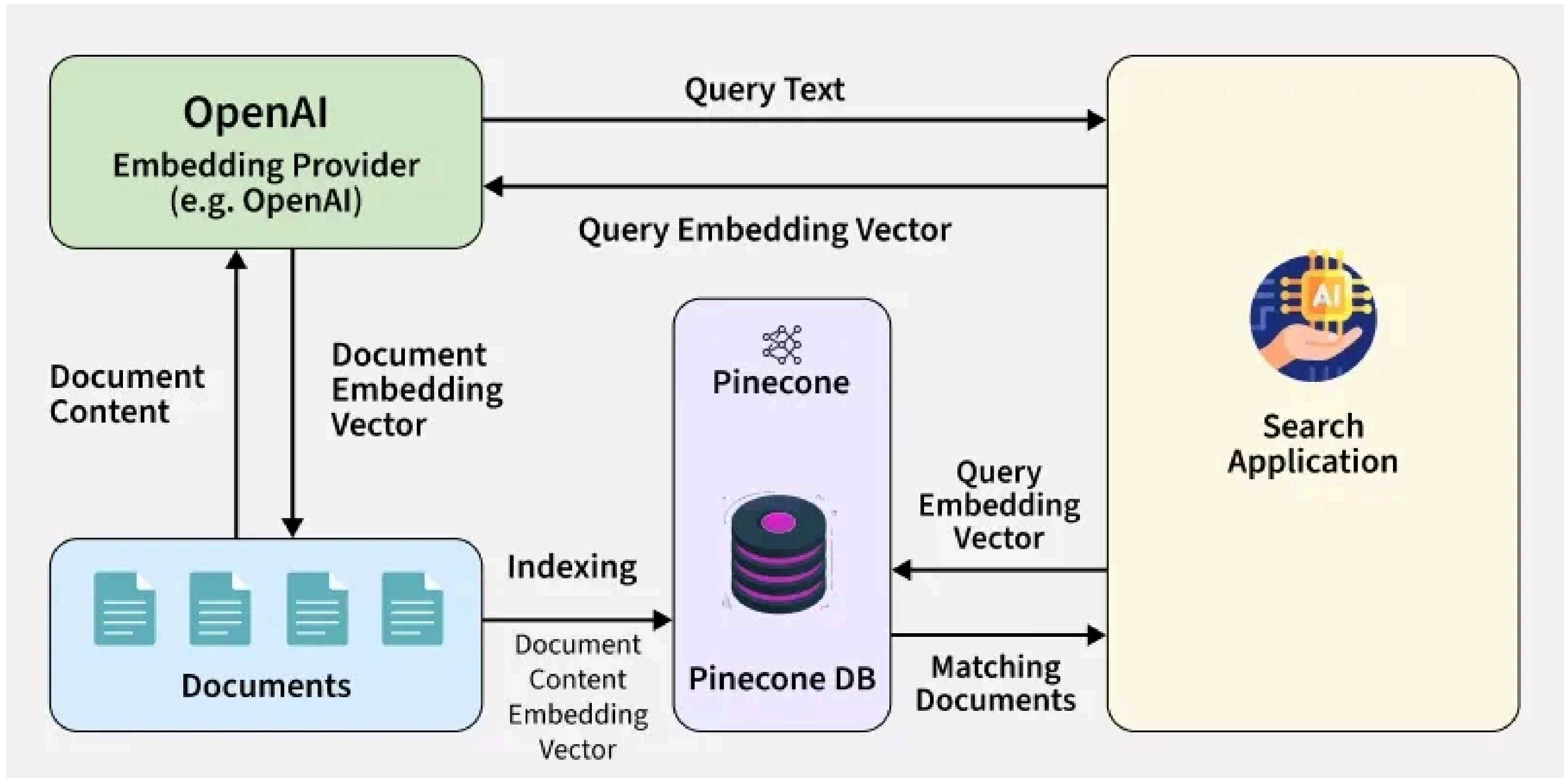
Code Walktrough

Q: How to Choose the Right Vector Database?



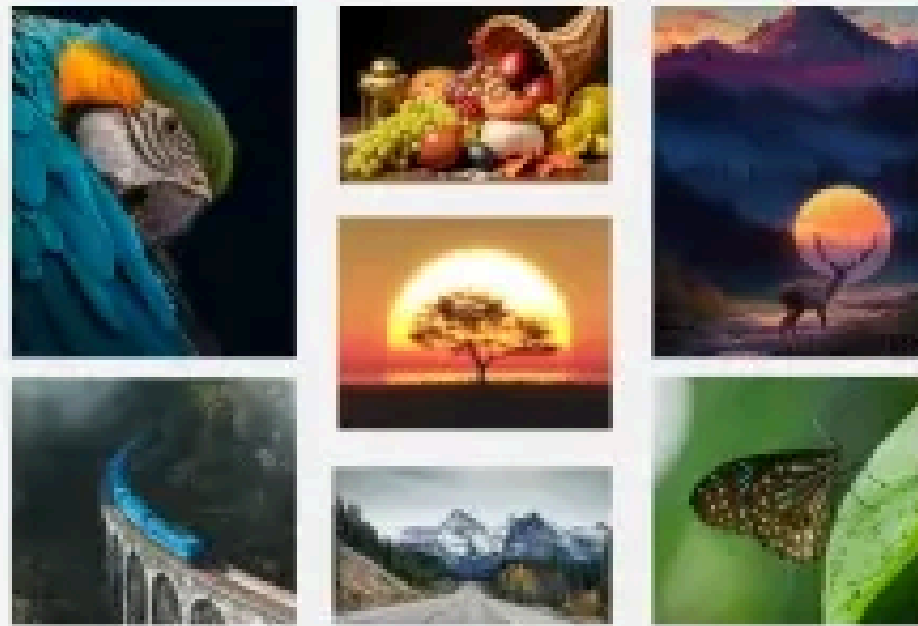
Popular Vector DBs

1. Pinecone

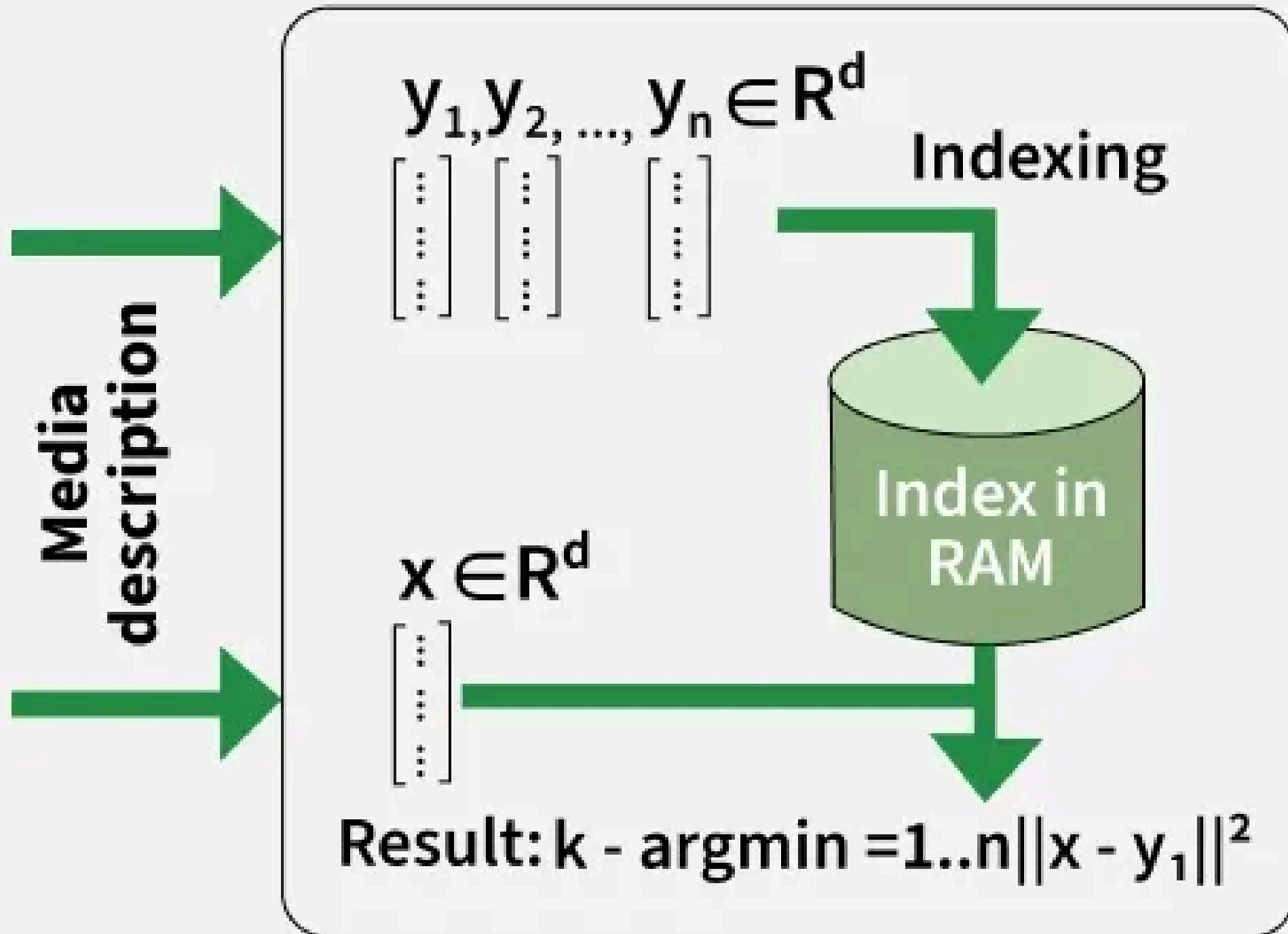
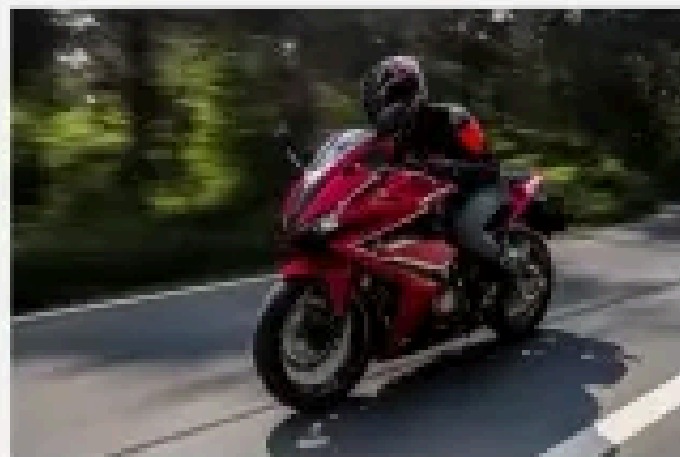


2. Faiss (Facebook AI Similarity Search)

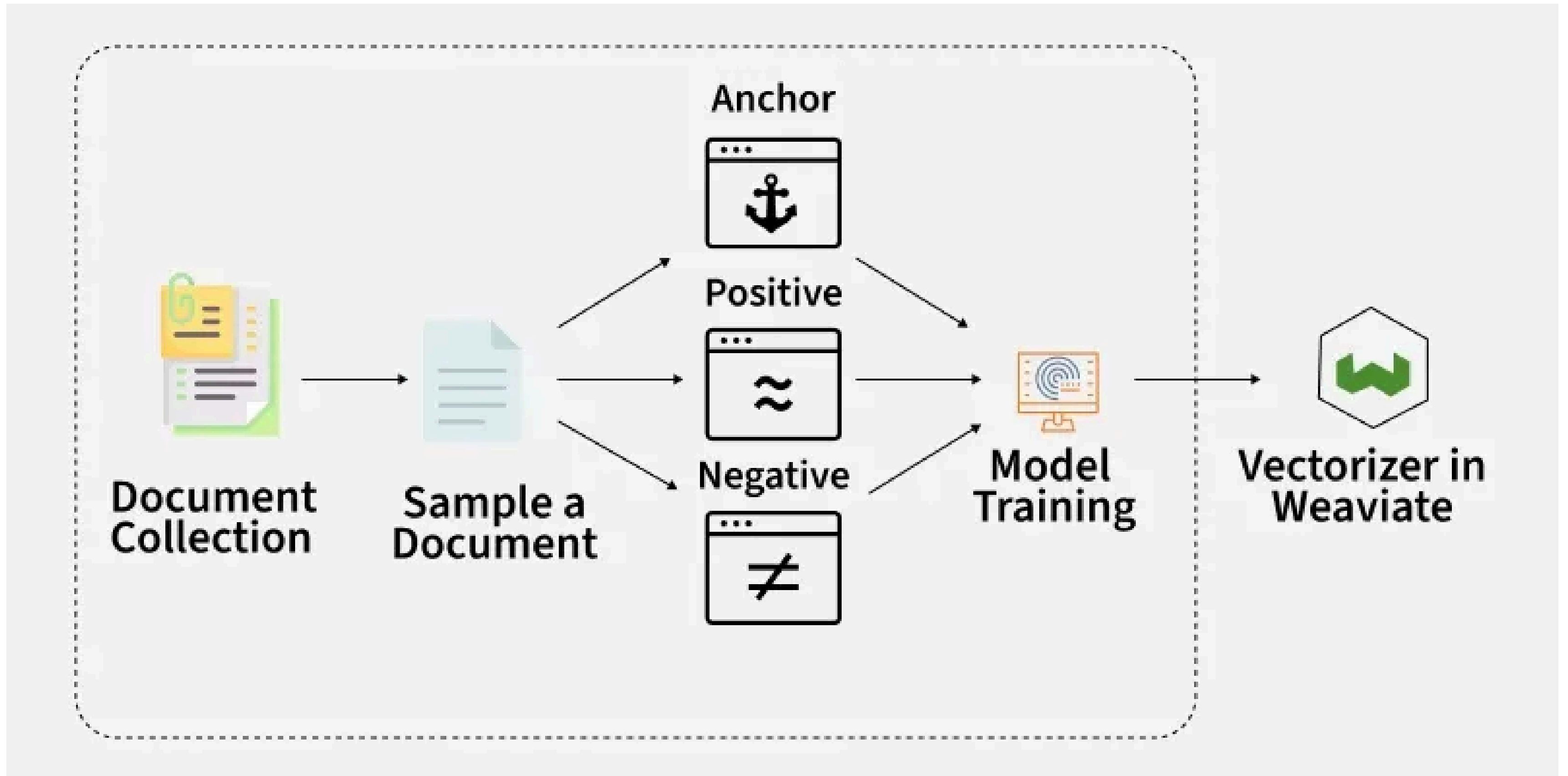
Build Index for a Collection:



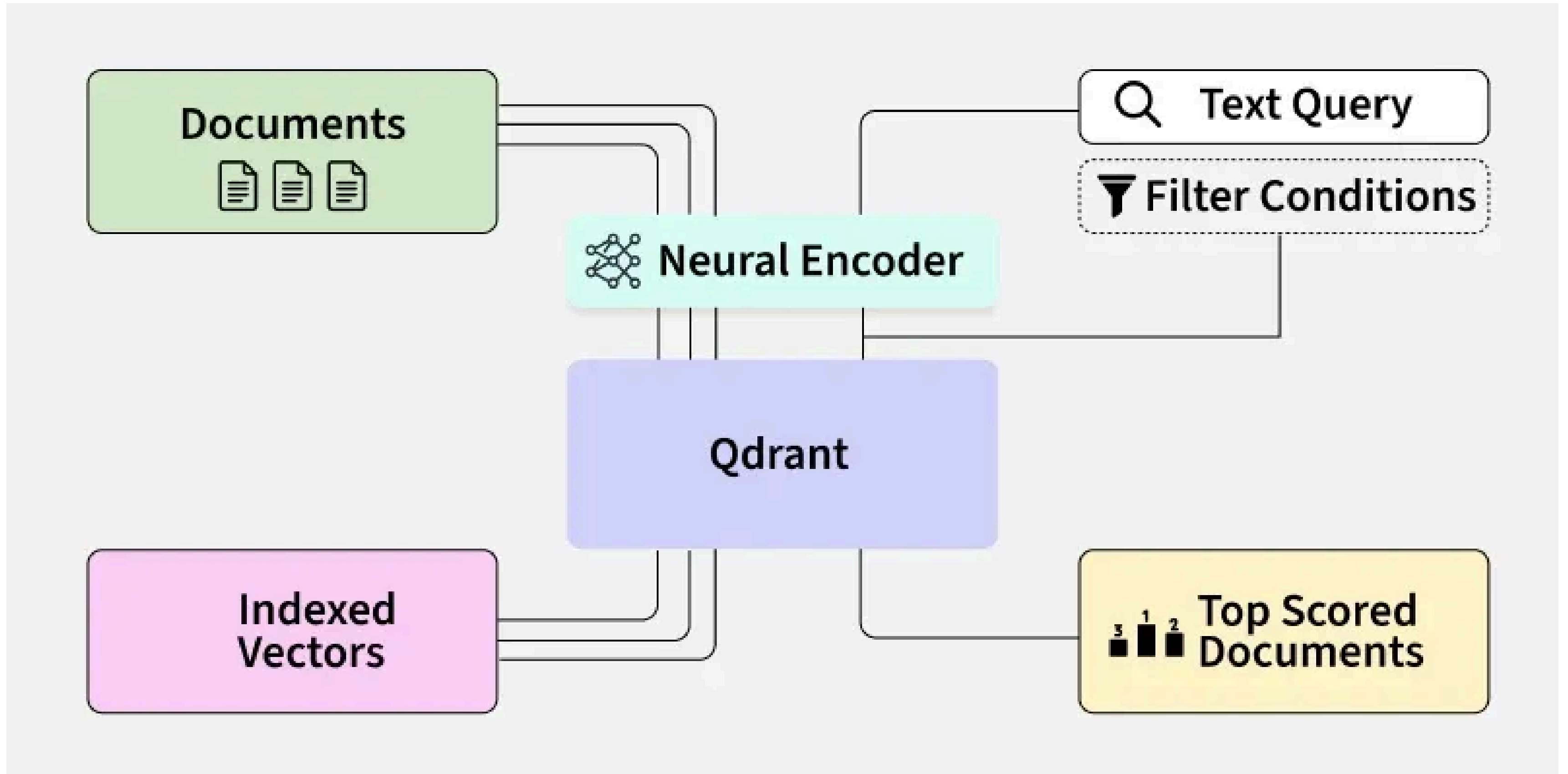
Query:



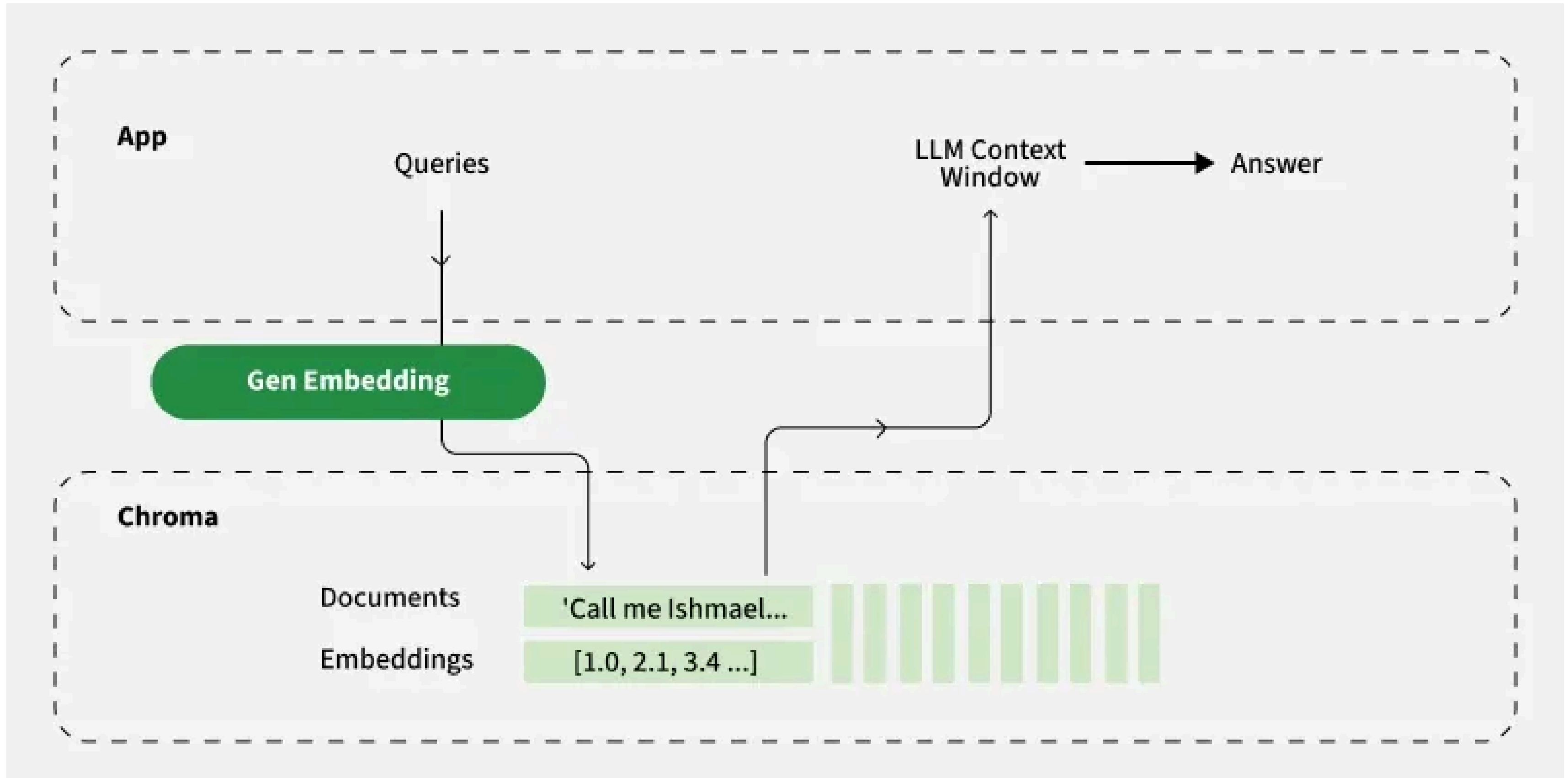
3. Weaviate



4. Qdrant



5. ChromaDB



Choosing the Right Vector Database for Our Use Case

- If we need speed and scalability: **Pinecone** and **ChromaDB** are great options for real-time performance at scale.
- If we want something open-source and customizable: **Faiss** and **Qdrant** are good choices.
- If our project involves large-scale text or semantic search: **Pinecone** and **Weaviate** are best for it.
- If we want simplicity: **Weaviate** is a user-friendly option with automatic machine learning model integration.

Code Walktrough



LANGCHAIN

Introduction to Langchain

- LangChain is an open-source framework designed to simplify the creation of applications using large language models (LLMs).
- It provides a standard interface for integrating with other tools and end-to-end chains for common applications.
- It helps AI developers connect LLMs such as GPT-4 with external data and computation.
- This framework comes for both Python and JavaScript.
- Key benefits include:
 - **Modular Workflow:** Simplifies chaining LLMs together for reusable and efficient workflows.
 - **Prompt Management:** Offers tools for effective prompt engineering and memory handling.
 - **Ease of Integration:** Streamlines the process of building LLM-powered application

Key Components of LangChain



Key Components of LangChain

- **Chains:** Chains define sequences of actions, where each step can involve querying an LLM, manipulating data or interacting with external tools. There are two types:
 - **Simple Chains:** A single LLM invocation.
 - **Multi-step Chains:** Multiple LLMs or actions combined, where each step can take the output from the previous step.
- **Prompt Management:** LangChain facilitates managing and customizing prompts passed to the LLM.
 - Developers can use **PromptTemplates** to define how inputs and outputs are formatted before being passed to the model.
 - It also simplifies tasks like handling dynamic variables and prompt engineering, making it easier to control the LLM's behavior.

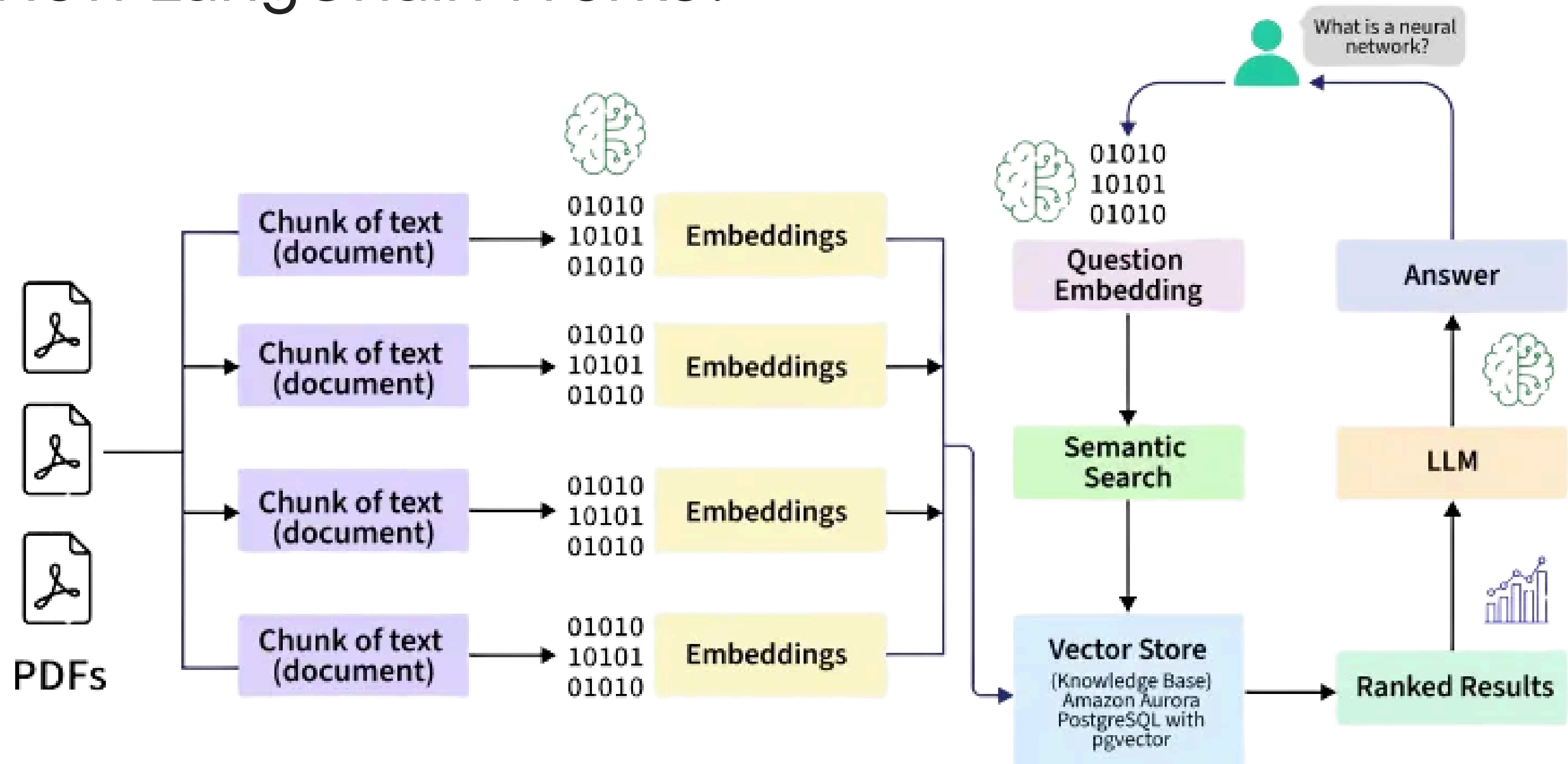
Key Components of LangChain

- **Agents:** Agents are autonomous systems within LangChain that take actions based on input data.
 - They can call external APIs or query databases dynamically, making decisions based on the situation.
 - These agents leverage LLMs for decision-making, allowing them to respond intelligently to changing input.
- **Vector Database:** LangChain integrates with a vector database which is used to store and search high-dimensional vector representations of data.
 - This is important for performing similarity searches, where the LLM converts a query into a vector and compares it against the vectors in the database to retrieve relevant information.

Key Components of LangChain

- **Models:** LangChain is model-agnostic meaning it can integrate with different LLMs such as OpenAI's GPT, Hugging Face models, DeepSeek R1 and more.
 - This flexibility allows developers to choose the best model for their use case while benefiting from LangChain's architecture.
- **Memory Management:** LangChain supports memory management allowing the LLM to "remember" context from previous interactions.
 - This is especially useful for creating conversational agents that need context across multiple inputs.
 - The memory allows the model to handle sequential conversations, keeping track of prior exchanges to ensure the system responds appropriately.

How LangChain Works?



How LangChain Works?

- **User Query**

- The process begins when a user submits a query or request.
- For example, a user might ask, “What’s the weather like today?” This query serves as the input to the LangChain pipeline.

- **Vector Representation and Similarity Search**

- Once the query is received, LangChain converts it into a vector representation using embeddings. This vector captures the semantic meaning of the query.
- The vector is then used to perform a similarity search in a vector database. The goal is to find the most relevant information or context stored in the database that matches the user's query.

How LangChain Works?

- **Fetching Relevant Information**

- Based on the similarity search, LangChain retrieves the most relevant data or context from the database.
- This step ensures that the language model has access to accurate and contextually appropriate information to generate a meaningful response.

- **Generating a Response**

- The retrieved information is passed to the language model (e.g., OpenAI's GPT, Anthropic's Claude or others). The LLM processes the input and generates a response or takes an action based on the provided data.
- For example, if the query is about the weather, the LLM might generate a response like, “Today’s weather is sunny with a high of 75°F.”
- The formatted response is returned to the user as the final output. The user receives a clear, accurate and contextually relevant answer to their query.

Code Walktrough

For detailed Langchain insides

<https://www.geeksforgeeks.org/artificial-intelligence/the-complete-langchain-ecosystem/>

Practical Applications

- Building Chatbots with LangChain
- Knowledge Graphs
- Build RAG pipeline using Open Source Large Language Models
- Chatbot Webapp with LangChain
- Building a Basic PDF Summarizer LLM Application
- Building an AI application with LlamaIndex
- RAG(Retrieval-Augmented Generation) using LLama3
- Make your Documents Gen AI-ready

Code Walkthrough

Mini Chat App



ECOSYSTEM OVERVIEW

